3
Privileges

In the previous chapter, you learned how to use Devise authentication features such as user session and registration management (sign-in, sign-up, sign-out, and so on).

After users are logged in, you will want to make sure that they can only access pages and page elements that they are supposed to see. You will want to define access control rules or privileges so that users cannot see protected resources, such as other users' private posts. The process of applying the rules in our web application is called **authorization**.

In Rails apps, the **CanCan gem** (`https://github.com/ryanb/cancan`) can be used for authorization by defining and applying privileges of what users can or cannot do. At the same time, Devise will still be used for authentication.

Let's take an example web application that uses CanCan together with Devise. After that, we will discuss more details on using CanCan, as well as testing and debugging it.

CollabBlogs – a web application for collaborative writing

Imagine that we are going to build a Rails web application that facilitates collaborative writing. Let's call it CollabBlogs. Assume that its basic functionalities are as follows:

- Guest users (not logged-in users) can read non-restricted posts.
- Only logged-in users can create posts.
- An administrator user can do anything he/she wants.
- A user can delete his/her own posts; he/she cannot delete posts created by other users.

www.it-ebooks.info